36. Online Chess (Real-time Multiplayer, Lichess-style)

> **Interviewer:** Design the backend for an online chess site — think Lichess or Chess.com. A player taps "Play" and gets matched against someone of similar strength. The two of them then play a live game with a **chess clock** (say 5 minutes each, blitz) — moves must show up on the opponent's board almost instantly. Spectators can watch popular games. When the game ends, both players' ratings update. Design it.

Candidate: Fun one — this isn't really a "scale the reads" or "scale the writes" problem in the usual CRUD sense. It's a **stateful, real-time, two-party session with a fairness clock**, sitting on top of a **matchmaking** problem and a **persistence** problem. Let me clarify before I commit to an architecture.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Matchmaking — pure rating-based pairing (like ELO/Glicko), or can players challenge a specific friend directly? 2. Time control — is it always a clock (blitz/rapid/classical), or are untimed "correspondence" games in scope too? 3. If a player's WiFi drops for 10 seconds mid-game, do we auto-forfeit them, or is there a grace period? 4. Do we need to actually **enforce chess rules** server-side (legal moves, check, checkmate, stalemate, draws), or can we trust the client and just relay bytes? 5. Spectators — how many can watch one game? Is a top game potentially watched by thousands?
>
> **Interviewer:** Rating-based matchmaking for now (friend challenges are a nice-to-have, skip the design detail). Focus on **timed games** — that's the hard part. Yes, a short grace period on disconnect before forfeiting. And absolutely: **the server must be the sole authority on legality and time** — we cannot trust the client, this is a competitive game with real ratings on the line. Spectators: yes, a popular game could have thousands of watchers.

Candidate: Good, that sharpens it. Let me write requirements.

Functional - `Play` → enqueue for matchmaking, get paired with an opponent of similar rating. - Live game: both players connect, exchange moves in real time, clock ticks per player. - Server validates every move (legality, whose turn) and enforces the clock (timeout = loss). - Resignation, draw offers, disconnect/reconnect handling. - Spectators can watch a live game (read-only stream of moves). - On game end: persist the game (PGN), update both players' ratings.

Non-functional - **Server-authoritative correctness:** a client must never be able to force an illegal move or manipulate its own clock. This is a trust boundary, not just a data-integrity nicety. - **Low latency, bidirectional:** a move must reach the opponent in well under a second, or the game *feels* broken — this is the "flash sale" of latency requirements, not throughput. - **Fairness under network failure:** whose clock runs during a disconnect must be unambiguous and identical on both players' screens. - **Stateful,**

not stateless: unlike a typical web CRUD service, a "game" is a long-lived session that must always be handled by *the same* process for its duration. - **Scale:** hundreds of thousands of concurrent games, spectator fan-out on hot games.

---

2. Estimation — the numbers that shape the design

> **Candidate:** Let me size this the way Lichess actually runs, because the numbers tell me where the pressure is.

```
Assume ~10M registered users, peak concurrent players ~500k, of which
~100,000 concurrent GAMES are in progress at peak (200k players, roughly).
Add spectators on popular games: say another 50k–100k watching connections.

=> ~250k–300k persistent WebSocket connections open at peak.

Move rate (this is the real pressure, like the burger giveaway's QPS):
  Blitz game (5 min/side): a player moves roughly every 3–6 seconds.
  Per GAME, combining both sides, call it ~1 move every ~4 seconds.
  100,000 concurrent games x (1 move / 4s) ≈ 25,000 moves/sec system-wide,
  each of which must be validated, clock-adjusted, and PUSHED to the
  opponent (and any spectators) in well under 100–200ms.

Finished games: ~5M games/day globally -> ~58 games/sec average completion.
Each finished game: PGN + metadata ≈ 1–3 KB. 5M x 2KB ≈ 10 GB/day. Trivial
storage. The bottleneck is NOT storage — it's stateful low-latency delivery
of 25k moves/sec to the exactly-right two (or few thousand) sockets.
```

Candidate: So the core challenge is **not** "big data," it's **"25,000 tiny, ordered, latency-sensitive, server-validated writes per second, each of which must be pushed instantly to a small, precise set of live connections, with a fairness clock riding on top."** That points straight at **stateful WebSocket sessions**, not stateless request/response.

---

3. A simple V1

> **Candidate:** Let me sketch the smallest thing that's *correct*, then scale it.

One game server process owns the game: the board (as FEN), the move list, both players' remaining clock time, and whose turn it is. Both players open a WebSocket to *that* process. A move arrives → the server validates it against chess rules, updates the clock based on server-side elapsed time, and **pushes** the resulting position to both sockets. At game end, it writes the PGN and result to Postgres and computes new ratings.

This already gets the core idea right: **one authoritative process per game**, **push not poll**. The gaps: routing 200k players to *the right* game server, surviving a server crash without losing games, matchmaking at scale, and spectator fan-out without hammering the game server. Let's scale it up.

---

4. Scaling up: matchmaking, many servers, multi-region

> **Interviewer:** Fine for one game. Now: 100,000 concurrent games, players constantly queuing and finishing, and users on three continents. Go.

4a. Matchmaking — pair by rating, widen the band over time

Candidate: Matchmaking is its own mini system, separate from any single game:

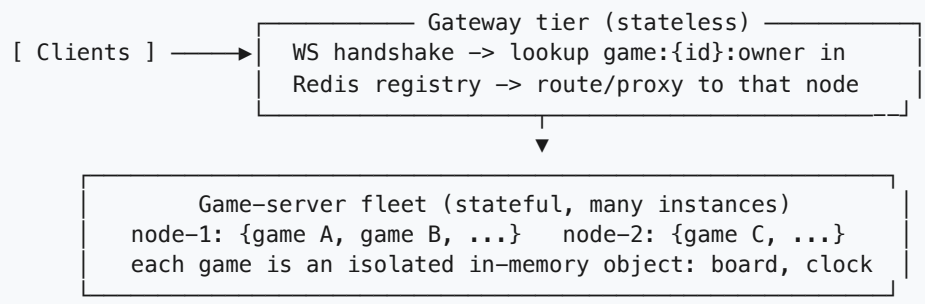
```
POST /matchmaking/enqueue {rating: 1500, timeControl: "5+0"}
      │
      ▼
[ Redis sorted set per (timeControl) pool, score = rating ]
      ZADD pool:blitz  1500  userId

[ Matchmaker loop, runs continuously per pool ]
  - pop a waiting player, ZRANGEBYSCORE around their rating ± band
  - band starts narrow (±25) and WIDENS the longer they wait (±50, ±100,
    ±400 after 30s) — guarantees everyone eventually gets a match, while
    keeping most pairings close in skill
  - on a hit: remove both from the pool, hand off to game-server allocator
```

A **Redis sorted set** is the right primitive: `ZRANGEBYSCORE` around a rating is exactly "find someone near my score," it's O(log n), and the queue is inherently ephemeral — nobody needs matchmaking history to survive a restart. Separate pools per time-control (bullet, blitz, rapid) so a 1-minute bullet player never gets matched into a 30-minute classical queue.

4b. Allocating a game server and registering it

Once two players are matched, an **allocator** picks a game-server instance with spare capacity (each instance can hold thousands of lightweight in-memory game objects) and spins up a new game object there, writing a routing entry: `Redis: game:{gameId}:owner = serverId-42` (TTL + heartbeat refresh). This is the **session registry** — the single source of truth for "which physical server currently owns this game's authoritative state." Every WebSocket connect for that `gameId` (player or spectator) is routed through the gateway, which looks up this key and proxies the connection there. This is what makes the system stateful-but-scalable: **any gateway node can accept the connection, but exactly one backend node owns the game.**

4c. Many concurrent games — horizontal, not vertical



Because each game is small and independent, this scales horizontally just by adding game-server nodes — no game ever needs more than one node, and one node holds thousands of unrelated games, so there's no cross-game contention.

4d. Multi-region

> **Interviewer:** Player A is in Tokyo, player B is in São Paulo. Where does the game live?

Candidate: A single game needs **one** authoritative owner, but two players on opposite sides of the planet can't both get a local server. Matchmaking should **bias toward same-region pairing first** (widen the rating band within a region before crossing regions), and when a cross-region match is unavoidable, place the game server in whichever region minimizes the **worse of the two** round-trips. We optimize for the max, not the average — a laggy connection for either side breaks fairness. Finished-game storage and rating updates, by contrast, have no latency pressure and replicate globally the normal way (regional Postgres replicas / a global rating service).

---

5. The game server: authoritative state, move validation, the clock

> **Interviewer:** Walk me through what actually happens inside that game-server process when a move arrives.

Candidate: This is the trust boundary, so the server does three things, in order, on every incoming move:

```
client sends: { type: "move", from: "e2", to: "e4", seq: 17 }
      |
      ▼
1. SEQUENCE CHECK — is seq the next expected number from THIS player?
   (rejects duplicates/replays and out-of-order delivery, see Deep-dive B)
      |
2. LEGALITY CHECK — run the move through a server-side chess rules engine:
   - is it this player's turn?
   - is the move legal for that piece given the current board (pins,
     checks, en-passant, castling rights, promotion, etc.)?
   - does it leave the mover's own king in check? (illegal)
   - reject with an error frame if not; the board never changes.
      |
3. CLOCK UPDATE — using the SERVER's wall-clock, compute elapsed time
   since this player's clock last started, decrement it. If it had
   already hit zero before this move arrived -> TIMEOUT LOSS, game over,
   the move is irrelevant.
      |
      ▼
4. Apply move to in-memory board + move list. Flip "whose turn."
5. Broadcast the new state to BOTH players' sockets AND the spectator
   fanout channel, push-style (no polling).
6. Append the move, synchronously, to a durable per-game move log
   (Redis stream / Kafka partition keyed by gameId) — see §8 (async).
```

Why this must all happen on the server: if the client decided legality, a modified client could play illegal moves (teleport a queen). If the client reported its own remaining time, a modified client could freeze its clock. **Server-authoritative is not a nice-to-have here — it is the entire integrity model of a competitive game.** (Deep-dive A goes deeper on this.)

---

6. Move relay, spectators, and reconnection

Move relay: the two players' WebSockets are held open for the whole game; the server pushes each validated move to both, plus a clock timestamp (clients tick the countdown down locally between updates, but every *authoritative* number comes from the server at move time — this avoids a steady stream of "clock: 61.2s, 61.1s..." messages).

Spectators: popular games can have thousands of watchers. If the game server pushed to every spectator socket directly, one viral game could overwhelm the very process that also needs to keep validating moves at low latency for the two actual players. Instead:

```
  [ Game server ] --publish move once--> [ Pub/Sub fanout tier (Redis
                                           Pub/Sub or a dedicated
                                           broadcast service) ]
                                                  |
                                    fan out to N thousand spectator sockets
                                    (a separate, horizontally-scaled tier)
```

This decouples **move authority** (must stay fast and correct for exactly 2 players) from **move broadcast** (can be delayed a few hundred ms, can be dropped and resubscribed, and can scale independently to any number of watchers).

Reconnection: if a player's socket drops, the gateway/registry still shows their game node; on reconnect, the client re-opens a WebSocket for the same `gameId`, the game server replies with the **full**

current state (board, move list, both clocks, whose turn) so the client's view is trivially resynced — no incremental patching needed. (Whether their clock kept running during the gap is the subject of Deep-dive B.)

---

7. API + data model

API (indicative)

```
POST /matchmaking/enqueue        { rating, timeControl }         -> queued
WS   /game/{id}                  bidirectional move/clock/state stream
GET  /game/{id}/state            snapshot (reconnect / spectator bootstrap)
POST /game/{id}/resign
POST /game/{id}/draw-offer | draw-accept
GET  /games/{id}/pgn             finished game export
GET  /users/{id}/rating-history
```

Data model — two very different lifetimes, two very different stores

Live game state lives **in the game server's memory** — board, move list, clocks — because it must mutate at sub-millisecond speed with zero network hop. Every move is also mirrored, fire-and-forget, into a durable **move log** (Redis stream / Kafka, keyed by `gameId`) purely so a new server can *replay and rebuild* the state if the owning process crashes (§9). The live state itself is **never** the source of truth in a database — it's ephemeral by nature.

Finished games + ratings (durable, queryable forever) go into **PostgreSQL**:

```
games(id, white_id, black_id, time_control, result, pgn_text,
      white_rating_before, black_rating_before,
      white_rating_after,  black_rating_after,
      started_at, ended_at)

users(id, rating, rating_deviation, games_played, ...)
```

Why relational, not a NoSQL/wide-column store, for finished games and ratings? Two reasons. First, a **rating update is a transactional read-modify-write across two different rows** (Elo: both players' new ratings depend on both old ratings and the result) — that's exactly the ACID transaction Postgres is built for; doing this without a transaction risks a lost-update race if both rating writes aren't atomic together. Second, the query patterns — "this user's game history," "leaderboard," "head-to-head record" — are relational joins, not huge-volume key-value lookups. The *write volume* (5M games/day, ~58/sec) is trivial for Postgres; this is nothing like the burger/flash-sale write-spike problem. If this were Lichess-at-its-biggest scale, you'd shard by user_id or add read replicas for history queries, but you would not reach for a NoSQL store just for throughput — the transactional rating update is the deciding factor, not raw volume.

---

8. Latency, consistency, async, caching

Latency: the design is built around push, not pull. A move's round-trip is: client → gateway (already connected, no new TCP/TLS) → game server (validate, in-memory, no disk) → push to opponent. Every

hop is memory-speed except the network itself — this is how we hit "well under a second" even at 25k moves/sec system-wide, because that load is spread across thousands of independent, tiny, isolated game objects — no shared hot resource to contend on (unlike the burger counter).

Consistency: the live game state is **strong, single-writer** — exactly one process owns a given game and processes its moves serially, so there's no distributed-consensus problem *during* the game at all; correctness comes from "only one authority exists," not locking or quorum. The rating update at game end is **strong and transactional** (both rows, one Postgres transaction). Everything else — the spectator view, leaderboard, search index, offline cheat scoring — is **eventual**, and that's fine; none of it needs to agree with the live game instant-by-instant.

Async (off the critical path): the durable move-log write happens alongside applying the move, never blocking the push to the opponent. Rating computation and the Postgres write happen after the game ends and the result is already shown — a slow DB write must never delay "you won." Cheat-detection analysis runs as an **offline batch job**, never inline with play.

Caching: a "last known state" cache (Redis) is updated on every move purely so a **new spectator** joining mid-game gets an instant snapshot without querying the game server directly. Finished-game reads (PGN downloads, profile pages) are cached/CDN'd normally — ordinary, boring traffic compared to the live-game path.

---

9. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Live authoritative game state	**In-process memory** on a dedicated game-server node, one object per game	Sub-millisecond mutation, single-writer means no locking/consensus needed. *Not* storing live state in Redis/Postgres as the primary copy on every move — that adds a network+serialization hop to the hottest, most latency-sensitive path in the whole system, purely to persist something that's disposable once the game ends.
Crash-recovery durability for live games	Append-only **move log** (Redis stream / Kafka, keyed by `gameId`) written alongside each move	Cheap, async, replayable — a new server rebuilds a game by replaying its moves. *Not* synchronous DB writes per move — that would gate every move's latency on disk I/O for a game that might last 40 moves and 5 minutes total.
Move relay to players/spectators	**WebSocket** (persistent, bidirectional)	Server pushes a move the instant it's validated — no poll delay, no re-establishing a connection per move. *Not HTTP request/response or polling* — see Deep-dive A; at 25k moves/sec system-wide, polling multiplies request volume by whatever poll frequency you'd need for a "live" feel, and still adds poll-interval latency on top.
Connection routing (which node owns a game)	**Redis registry** (`game:{id}:owner`) + a routing/proxy gateway tier	O(1) lookup of "who owns this," heartbeat + TTL detect a dead owner quickly. *Not* a stateless round-robin LB — a game is inherently stateful; a new request for an existing game MUST land on the one node that has it in memory, unlike a normal web request that can hit any pod.

Concern	Choice	Why this, and why not the alternative
Matchmaking queue	**Redis sorted sets** per time-control pool, scored by rating	`ZRANGEBYSCORE` is exactly "find someone near my rating," O(log n), and the queue is transient by nature — nothing here needs to survive a restart. *Not* a DB table polled for matches — polling a table for "who's near my rating" under high queue churn is slower and adds needless durability for ephemeral data.
Spectator fan-out	**Pub/Sub broadcast tier** (Redis Pub/Sub or a dedicated fanout service), decoupled from the game server	A viral game with thousands of watchers must not compete with the two real players for the game server's CPU/socket budget. *Not* the game server pushing to every spectator socket itself — that couples "move authority" (must stay fast) to "broadcast scale" (must scale independently and can tolerate lag).
Finished games + ratings	**PostgreSQL** (ACID)	Rating updates are a transactional read-modify-write across two rows; game history is relational (joins, leaderboards). *Not* a NoSQL wide-column store chosen for throughput — finished-game write volume (~58/sec average) doesn't need that, and the transactional rating update is exactly what NoSQL eventual-consistency models make harder, not easier.
Cheat-detection pipeline	**Async, offline batch analysis** (Kafka of finished-game events → analysis workers)	Statistical engine-correlation checks are expensive and inherently retrospective — running them online would add latency to gameplay for a signal that only matters after the fact.

Say-it-out-loud justification: "*The live game lives in one process's memory because that's the only way to get sub-millisecond, single-writer authority with zero consensus overhead; WebSockets push moves instantly instead of making 25,000 moves/sec pay a polling tax; a Redis registry tells the stateless gateway tier which stateful node owns a given game; matchmaking and spectator fan-out are separate, horizontally-scalable concerns kept off the hot path; and once a game ends, it becomes a completely different, much calmer problem — an ACID transaction in Postgres for the rating update and a durable row for history.*"

---

10. Failure modes & graceful degradation

Failure	What happens	Mitigation
Game-server node crashes mid-game	In-memory state for every game it held is gone	Replay the durable move log on a freshly allocated node to rebuild board/clock/move-list; freeze both players' clocks during the outage so neither is penalized for the crash (see Deep-dive B)
Redis registry down	New WebSocket connects/reconnects can't be routed; matchmaking pairing pauses	Registry runs replicated with failover; games *already* connected are unaffected (state lives on the game server, not in Redis); gateway can cache recent lookups briefly

Failure	What happens	Mitigation
Pub/Sub fanout tier overloaded	Spectators see stale/delayed positions	Shed spectator load first (lower fanout rate, "reconnect" prompt) — players are on a separate path and stay unaffected
Postgres down at game end	Rating/history write delayed	Result is already shown to players client-side (the game server knows the outcome); finish-events queue durably (Kafka) and drain into Postgres on recovery — no game result is lost
Player's network blips	Risk of unfair clock loss or move desync	Grace/reconnect window before declaring timeout-loss; sequence numbers reject stale/duplicate moves on resume (full detail in Deep-dive B)
Bot / engine-assisted cheating	Server can't detect this via move legality alone	Offline statistical analysis (move-quality vs. engine correlation, timing patterns) flags accounts for review; never auto-ban on a single game without human review to avoid false positives

11. Follow-up curveballs

> **Interviewer:** Two players are matched, but one of them closed the tab a second before the game actually started. What happens?

Candidate: Don't commit the match until **both** sides acknowledge. The allocator sends a "game found" handshake to both clients and waits briefly for both to ack (connect their WebSocket); if one fails to show up within a short window, that player is penalized lightly (or just dropped) and the other is put back at the front of the matchmaking queue rather than being stuck. We never want a "game" to exist with only one real participant.

> **Interviewer:** A hugely popular streamer's game gets 50,000 spectators. Does anything break?

Candidate: The game server itself is unaffected — it only ever talks to 2 players and publishes once to the fanout tier per move. The pressure lands entirely on the **pub/sub broadcast tier**, which is a stateless, horizontally-scalable fan-out problem (the same shape as a CDN edge distributing one video stream to many viewers) — add fanout capacity, not game-server capacity. This is exactly why we split the two concerns in §6.

> **Interviewer:** How do ratings actually get updated?

Candidate: Standard Elo (or Glicko-2 if we want a confidence/deviation term): given both players' current ratings and the result, compute each new rating with a K-factor, and write both rows in a single Postgres transaction so there's no window where one player's rating updated and the other's didn't. This can run inline at game-end (it's cheap arithmetic) or be handed to a small rating service — either way, it's not on the *move* path, only the *game-end* path, so it never competes with the 25k moves/sec.

> **Interviewer:** Someone claims they were winning when they got disconnected and unfairly lost on time. How do you investigate?

Candidate: Because every move and clock event is server-timestamped and durably logged (§7), we can reconstruct the exact sequence: when the disconnect was detected, when the grace window expired, what each clock read at every step. The server's log is the single source of truth for a dispute — there's no "he said, she said" because the client was never trusted to report time in the first place.

---

12. Deep-dive: "Why not trust the client, and why not stateless HTTP polling instead of stateful WebSocket game servers?"

> **Interviewer:** All this stateful-server, registry, WebSocket machinery is a lot of moving parts. Why not the simple thing: the client keeps its own board state and clock, calls `POST /move` when it moves, and other clients `GET /game/{id}` every second to see what changed? Stateless, cacheable, boring, and boring is usually good.

Candidate: It's the natural first instinct — most CRUD systems *should* be stateless — but chess has two properties that break both halves of that proposal: it's **adversarial** (the other party has an incentive to cheat) and it's **latency-and-fairness-critical** (a clock is being enforced between two mutually distrustful parties). Let me take them separately.

Why not client-authoritative move validation / clock?

If the client decides what's legal and what time is left, a modified client can play a move the rules don't allow (an illegal "escape" from checkmate), report its own clock as never running out, or silently take back an illegal move and only submit the legal-looking end state. None of this requires sophisticated hacking — browser dev tools or a patched client are enough, because **there is nothing external checking the client's claims.** Rated competitive play makes this a real integrity failure, not a cosmetic bug — ratings and rankings are downstream of "the server believed the client."

Why not stateless HTTP request/response (polling) instead of WebSocket?

Two separate problems, both fatal at this system's scale:

1. **Latency.** A move must reach the opponent in well under a second for the game to feel live. Polling `GET /game/{id}` achieves that only by polling *very* frequently (every 200-500ms) — from **every one of ~250k-300k open sessions** simultaneously, almost all of which return "nothing changed." That's an enormous amount of wasted request volume just to approximate what a push channel gives you for free, and it still adds a poll-interval's worth of extra lag on top of network latency.
2. **Statelessness doesn't actually fit the workload.** A "stateless" HTTP design still needs to know *where* the current board is before it can answer a `GET` — so you'd end up building the exact same "which node owns this game" registry anyway, just without the benefit of a push channel. Stateless HTTP is a great fit when any node can answer any request from shared durable storage;

here, the authoritative state is *not* shared — it's owned by exactly one process for low-latency mutation.

```
Client-authoritative                    Server-authoritative (chosen)
────────────────────                    ─────────────────────────────
client enforces rules/clock             server enforces rules/clock
trivially cheatable                     cheat-proof by construction
(patched client = win every game)       (client can only ever propose)

HTTP polling                            WebSocket, stateful game server
────────────────────                    ─────────────────────────────
"any node can answer" — but there's     one owning node per game, push
no shared source of truth to poll        the instant a move is validated
without rebuilding a registry anyway
poll interval = extra latency on top    push = latency is just network RTT
N sessions x poll frequency = huge      1 event per move, fanned out only
wasted request volume                    to the parties who need it
```

Concern	Client-authoritative + HTTP polling	Server-authoritative + stateful WebSocket
Cheating	Trivial — patch the client, it's the only judge	Impossible via move-forgery — server is sole judge of legality and time
Latency	Bounded below by poll interval; frequent polling to compensate is expensive at scale	Push-based; latency ≈ network RTT, no artificial floor
Load at scale	N open sessions × poll frequency, mostly wasted "nothing changed" responses	1 event per validated move, fanned out only to interested parties
Where state lives	Ambiguous — "stateless" servers still need *some* shared source of truth to answer GETs, so you rebuild routing anyway	Explicit and simple — one process per game, a registry says which one
Fits which workload	Great for CRUD where any replica can serve from shared durable storage	Great for a small number of mutually-distrustful parties needing instant, ordered, authoritative updates

> **Rule of thumb:** *When two parties don't fully trust each other and a fairness clock is involved, the server must be the sole judge of both rules and time — never the client. And when updates must reach a small, precise set of live parties in well under a second, push beats poll every time; the "cost" of a stateful WebSocket session is the price of that latency, and you'd end up half-building the same routing layer under a "stateless" polling design anyway, just slower.*

---

13. Deep-dive: the chess-clock + disconnect race

> **Interviewer:** Here's the nasty one. A player's WiFi drops for a few seconds mid-game. Whose clock is running during that gap? What if it drops for much longer — do we forfeit them? And separately: what stops a laggy client from sending two moves that arrive out of order, or a move for the wrong turn?

Candidate: This is the sharpest fairness edge in the whole design, because — like the flash-sale reservation race — it's a collision between **independent clocks that don't know about each other**: the game's chess clock, the network's actual latency, and the client's own retry/reconnect behavior. Let me walk it through.

The baseline rule, stated precisely

The player's own clock runs during their own turn, period — disconnected or not. This sounds harsh but it's the *correct* and *simple* rule: in an over-the-board game, if you leave the table on your own turn, your clock still runs; a disconnect during *your own thinking time* is indistinguishable from just being slow, and the server has no way to tell "genuinely disconnected" from "AFK." Complexity only enters for the case below.

The timeline: disconnect during the OPPONENT's turn

```
t=0:00.0   It is Black's turn. White's clock is PAUSED (correct — not
           White's turn). Black is thinking.

t=0:00.5   Black's connection drops (WiFi blip). Black's clock keeps
           running — it's Black's turn, this is expected/correct so far.

t=0:04.0   White submits a move? No — wait, it's Black's turn, White
           can't move. Good, turn ownership already prevents this class
           of confusion by construction (§5, step 1–2).

t=0:08.0   Black is still disconnected. Black's clock has now burned 8s
           of real thinking time PLUS the disconnected time — that's
           fine, indistinguishable from slow play, per the baseline rule.

t=0:15.0   Black's clock would hit zero at, say, t=0:20 if this continues.
           Now the question sharpens: is this "Black is thinking slowly"
           or "Black's device died and they'll never move again"? The
           server CANNOT tell the difference from server-side signals
           alone within a few seconds — a genuine network hiccup and a
           dead phone look identical at t=15s.
```

The actual hazard isn't "whose clock runs" (that's simple — the mover's own clock always runs on their own turn). The hazard is a **short, recoverable network blip getting treated the same as a permanent abandonment**, either forfeiting a player unfairly fast, or leaving an opponent stuck waiting forever for someone who's genuinely gone.

Concurrent-move / out-of-turn handling (the other half of the question)

Turn ownership already rejects a move sent on the wrong turn (§5 step 1-2: the server checks whose turn it is before anything else). The remaining risk is **duplicate or out-of-order delivery** of the *same* player's own moves, e.g. a flaky connection causes the client to retry a move it thinks didn't send:

```
client sends: move seq=17 (network delays the ack)
client, seeing no ack, RETRIES:  move seq=17 (same move, resent)
server received seq=16 already applied. Expected next = 17.
  - first seq=17 arrives -> applied, ack sent, expected becomes 18
  - duplicate seq=17 arrives -> server sees "already have this seq,
    already acked" -> silently drop / re-send the cached ack, do NOT
    apply the move twice
```

Monotonic **sequence numbers per player**, checked at step 1 of move processing, make this a non-issue: the server always knows the next expected sequence number for each side and treats anything else (duplicate, stale, or out-of-order) as a no-op-with-cached-ack rather than a new move. This is the same idempotency-key pattern used for the burger claim and the flash-sale checkout — applied here per-move instead of per-request.

Ranked fixes for the disconnect/forfeit problem

1. **Server-authoritative clock (non-negotiable foundation).** Every clock decision comes from server-side timestamps, never a client-reported number. Prevents outright manipulation but doesn't yet resolve the "blip vs. abandonment" ambiguity — that needs the fixes below.
2. **A short reconnect grace window, separate from the chess clock itself.** On socket drop, start a **connection-grace timer** (e.g., 15-30s) independent of the chess clock. Reconnect within it and nothing special happened — the chess clock already ran fairly the whole time. Its real job is narrower than it sounds: it just stops the server from doing anything *drastic* (an immediate forfeit notice to the opponent) on a blip likely to self-heal.
3. **A disconnect is just one more way the clock runs out — don't invent a second forfeit path.** Resist adding a separate "abandonment forfeit" distinct from "timeout." Two parallel forfeit mechanisms are exactly how the two players' views end up disagreeing — one thinks "I lost on time," the other thinks "I won because they left." Keep a **single** rule (the clock) and let disconnection be one of the ways it runs out unattended.
4. **Reconnect always resyncs from a full server snapshot, never a diff.** Send the complete current state (board, move list, both clocks, whose turn) rather than replaying missed events — this is what actually guarantees both players' views of time agree: don't reconcile client-side clocks at all, always resync from the one source of truth.
5. **A long-disconnect inactivity forfeit, only for untimed/correspondence formats.** Fast time controls already resolve abandonment via the clock (fix #3). Days-long correspondence games need a separate, much longer inactivity timeout — a different mechanism for a different mode.

Recommended approach

Lead with **#1 and #3 together**: the clock is the single source of truth for abandonment, and disconnection is just one way it runs out — no separate forfeit path to keep in sync. Add **#2** purely to avoid overreacting to a blip that's likely to resolve in seconds. Add **#4** so reconnection is always a clean resync. Reserve **#5** for long-format games only.

> **What to say out loud:** "*The chess clock is the single source of truth for fairness — a disconnect is just one more reason a player's own clock might run out, not a separate kind of loss to invent and keep consistent with the clock. A short connection-grace window stops us from overreacting to a blip that self-heals in seconds, but it never overrides what the server-side clock actually says. And both players always resync from a full server snapshot on reconnect, rather than patching in missed events, so there is never a moment where the two sides' views of the clock or the board can disagree — the server is the only clock that matters, and everyone else is just displaying it.*"

Summary — the mental model

Online chess is **not a big-data problem — it's a "many small, stateful, mutually-distrustful, latency-critical sessions" problem.** The winning shape: **rating-based matchmaking in a Redis sorted-set queue**, a fleet of **stateful game-server processes** each holding a handful of games' authoritative state **in memory** (one owner per game, no locking needed because there's only one writer), a **Redis registry** so a stateless gateway tier can route connections to the right owning node, **WebSocket push** instead of polling so moves and clock updates arrive in well under a second, a decoupled **pub/sub fanout tier** so spectator load never touches the two real players' path, and a clean handoff at game-end into **ACID Postgres** for the durable game record and the transactional rating update. The server is the sole authority on both *rules* and *time* — that single decision is what makes a competitive, real-money-adjacent, adversarial multiplayer game trustworthy at all.